

Elementary Graph Algorithms I

Dr. Bugra Caskurlu¹

¹The School of Computing
New Uzbekistan University

November, 2025

Defining Directed Graphs

Definitions

- A **directed graph** G is a pair (V, E) , where V is a finite set and E is a binary relation on V .

Defining Directed Graphs

Definitions

- A **directed graph** G is a pair (V, E) , where V is a finite set and E is a binary relation on V .
- The set V is called the **vertex set** of G , and its elements are called vertices.

Defining Directed Graphs

Definitions

- A **directed graph** G is a pair (V, E) , where V is a finite set and E is a binary relation on V .
- The set V is called the **vertex set** of G , and its elements are called vertices.
- The set E is called the **edge set** of G , and its elements are called **edges**.

Defining Directed Graphs

Definitions

- A **directed graph** G is a pair (V, E) , where V is a finite set and E is a binary relation on V .
- The set V is called the **vertex set** of G , and its elements are called **vertices**.
- The set E is called the **edge set** of G , and its elements are called **edges**.

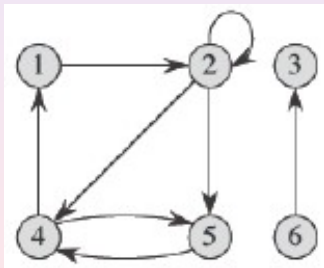


Figure: A pictorial representation of a directed graph on the vertex set $V = \{1, 2, 3, 4, 5, 6\}$. Vertices are represented by circles, and edges are represented by arrows. Note that self-loops (edges from a vertex to itself) are possible.

Defining Undirected Graphs

Definitions

- In an **undirected graph** $G = (V, E)$, the edge set E consists of unordered pairs of vertices, rather than ordered pairs. That is, an edge is a set $\{u, v\}$, where $u, v \in V$ and $u \neq v$.

Defining Undirected Graphs

Definitions

- In an **undirected graph** $G = (V, E)$, the edge set E consists of unordered pairs of vertices, rather than ordered pairs. That is, an edge is a set $\{u, v\}$, where $u, v \in V$ and $u \neq v$.
- By convention, we use the notation (u, v) for an edge, rather than the set notation $\{u, v\}$, and we consider (u, v) and (v, u) to be the same edge.

Defining Undirected Graphs

Definitions

- In an **undirected graph** $G = (V, E)$, the edge set E consists of unordered pairs of vertices, rather than ordered pairs. That is, an edge is a set $\{u, v\}$, where $u, v \in V$ and $u \neq v$.
- By convention, we use the notation (u, v) for an edge, rather than the set notation $\{u, v\}$, and we consider (u, v) and (v, u) to be the same edge.
- In an undirected graph, self-loops are forbidden, and so every edge consists of two distinct vertices.

Defining Undirected Graphs

Definitions

- In an **undirected graph** $G = (V, E)$, the edge set E consists of unordered pairs of vertices, rather than ordered pairs. That is, an edge is a set $\{u, v\}$, where $u, v \in V$ and $u \neq v$.
- By convention, we use the notation (u, v) for an edge, rather than the set notation $\{u, v\}$, and we consider (u, v) and (v, u) to be the same edge.
- In an undirected graph, self-loops are forbidden, and so every edge consists of two distinct vertices.

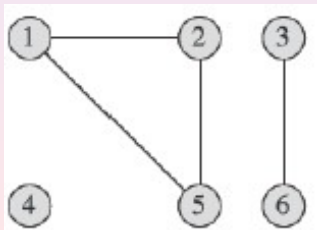


Figure: A pictorial representation of an undirected graph on the vertex set $V = \{1, 2, 3, 4, 5, 6\}$.

Defining Incidence

Definitions

- Many definitions for directed and undirected graphs are the same, although certain terms have slightly different meaning in the two contexts.

Defining Incidence

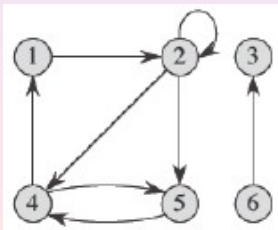
Definitions

- Many definitions for directed and undirected graphs are the same, although certain terms have slightly different meaning in the two contexts.
- If (u, v) is an edge in a directed graph $G = (V, E)$, we say that (u, v) is **incident from** or **leaves** vertex u and **incident to** or **enters** vertex v .

Defining Incidence

Definitions

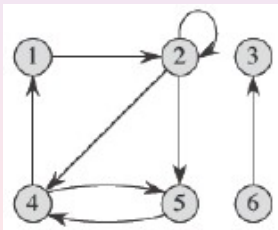
- Many definitions for directed and undirected graphs are the same, although certain terms have slightly different meaning in the two contexts.
- If (u, v) is an edge in a directed graph $G = (V, E)$, we say that (u, v) is **incident from** or **leaves** vertex u and **incident to** or **enters** vertex v .
- For example, the edges leaving vertex 2 in the figure are $(2, 2)$, $(2, 4)$, and $(2, 5)$. The edges entering vertex 2 are $(1, 2)$ and $(2, 2)$.



Defining Incidence

Definitions

- Many definitions for directed and undirected graphs are the same, although certain terms have slightly different meaning in the two contexts.
- If (u, v) is an edge in a directed graph $G = (V, E)$, we say that (u, v) is **incident from** or **leaves** vertex u and **incident to** or **enters** vertex v .
- For example, the edges leaving vertex 2 in the figure are $(2, 2)$, $(2, 4)$, and $(2, 5)$. The edges entering vertex 2 are $(1, 2)$ and $(2, 2)$.



Definition

If (u, v) is an edge in an undirected graph, we say that (u, v) is **incident on** vertices u and v .

Defining Adjacency

Definitions

- If (u, v) is an edge in a graph $G = (V, E)$, we say that vertex v is **adjacent** to vertex u .

Defining Adjacency

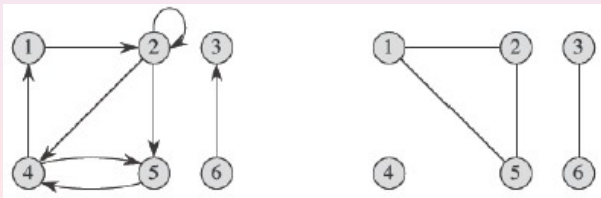
Definitions

- If (u, v) is an edge in a graph $G = (V, E)$, we say that vertex v is **adjacent** to vertex u .
- When the graph is undirected, the adjacency relation is symmetric. When the graph is directed, the adjacency relation is not necessarily symmetric.

Defining Adjacency

Definitions

- If (u, v) is an edge in a graph $G = (V, E)$, we say that vertex v is **adjacent** to vertex u .
- When the graph is undirected, the adjacency relation is symmetric. When the graph is directed, the adjacency relation is not necessarily symmetric.
- Vertex 2 is adjacent to vertex 1 in both graphs, since the edge $(1, 2)$ belongs to both graphs. Vertex 1 is not adjacent to vertex 2 on the left, since the edge $(2, 1)$ does not belong to the graph on the left.



Defining Degree

Definitions

- The **degree** of a vertex in an undirected graph is the number of edges incident on it. A vertex whose degree is 0, is **isolated**.

Defining Degree

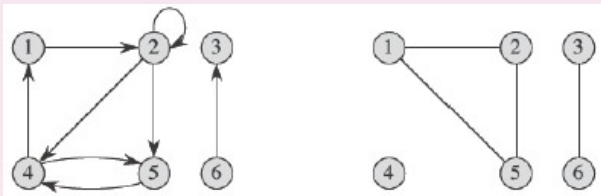
Definitions

- The **degree** of a vertex in an undirected graph is the number of edges incident on it. A vertex whose degree is 0, is **isolated**.
- In a directed graph, the **out-degree** of a vertex is the number of edges leaving it, and the **in-degree** of a vertex is the number of edges entering it.

Defining Degree

Definitions

- The **degree** of a vertex in an undirected graph is the number of edges incident on it. A vertex whose degree is 0, is **isolated**.
- In a directed graph, the **out-degree** of a vertex is the number of edges leaving it, and the **in-degree** of a vertex is the number of edges entering it.
- The **degree** of a vertex in a directed graph is its in-degree plus its out-degree. The degree of vertex 2 on the left is 5. Vertex 4 on the right is isolated.



Defining Paths

Definitions

- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0, u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.

Defining Paths

Definitions

- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0, u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.
- The length of the path is the number of edges in the path.

Defining Paths

Definitions

- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0, u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.
- The length of the path is the number of edges in the path.
- The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.

Defining Paths

Definitions

- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0$, $u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.
- The length of the path is the number of edges in the path.
- The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.
- There is always a 0-length path from u to u .

Defining Paths

Definitions

- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0, u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.
- The length of the path is the number of edges in the path.
- The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.
- There is always a 0-length path from u to u .
- If there is a path p from u to u' , we say that u' is **reachable** from u via p , which we sometimes write as $u \overset{p}{\rightsquigarrow} u'$ if G is directed.

Defining Paths

Definitions

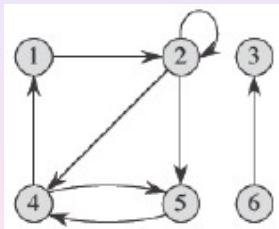
- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0, u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.
- The length of the path is the number of edges in the path.
- The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.
- There is always a 0-length path from u to u .
- If there is a path p from u to u' , we say that u' is **reachable** from u via p , which we sometimes write as $u \overset{p}{\rightsquigarrow} u'$ if G is directed.
- A path is **simple** if all vertices in the path are distinct.

Defining Paths

Definitions

- A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, \dots, v_k \rangle$ of vertices such that $u = v_0, u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$.
- The length of the path is the number of edges in the path.
- The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$.
- There is always a 0-length path from u to u .
- If there is a path p from u to u' , we say that u' is **reachable** from u via p , which we sometimes write as $u \overset{p}{\rightsquigarrow} u'$ if G is directed.
- A path is **simple** if all vertices in the path are distinct.
- A **subpath** of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is a contiguous subsequence of its vertices. That is, $0 \leq i \leq j \leq k$, the subsequence of vertices $\langle v_i, v_{i+1}, \dots, v_j \rangle$ is a subpath of p .

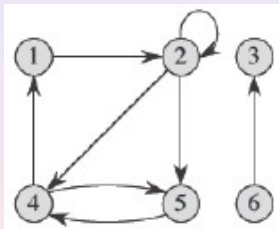
Path Examples



Explanation

- The path $\langle 1, 2, 5, 4 \rangle$ is a simple path of length 3.

Path Examples



Explanation

- The path $\langle 1, 2, 5, 4 \rangle$ is a simple path of length 3.
- The path $\langle 2, 5, 4, 5 \rangle$ is not simple.

Defining Cycles

Definitions

- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.

Defining Cycles

Definitions

- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.
- The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct.

Defining Cycles

Definitions

- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.
- The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct.
- A self-loop is a simple cycle of length 1.

Defining Cycles

Definitions

- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.
- The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct.
- A self-loop is a simple cycle of length 1.
- Two paths $\langle v_0, v_1, \dots, v_{k-1}, v_0 \rangle$ and $\langle v'_0, v'_1, \dots, v'_{k-1}, v'_0 \rangle$ form the same cycle if there exists an integer j such that $v'_i = v_{i+j \bmod k}$ for $i = 0, 1, \dots, k-1$.

Defining Cycles

Definitions

- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.
- The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct.
- A self-loop is a simple cycle of length 1.
- Two paths $\langle v_0, v_1, \dots, v_{k-1}, v_0 \rangle$ and $\langle v'_0, v'_1, \dots, v'_{k-1}, v'_0 \rangle$ form the same cycle if there exists an integer j such that $v'_i = v_{i+j \bmod k}$ for $i = 0, 1, \dots, k-1$.
- A directed graph with no self-loops is **simple**.

Defining Cycles

Definitions

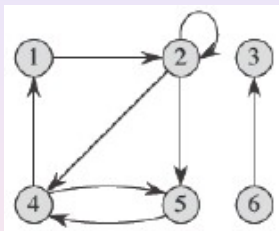
- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.
- The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct.
- A self-loop is a simple cycle of length 1.
- Two paths $\langle v_0, v_1, \dots, v_{k-1}, v_0 \rangle$ and $\langle v'_0, v'_1, \dots, v'_{k-1}, v'_0 \rangle$ form the same cycle if there exists an integer j such that $v'_i = v_{i+j \bmod k}$ for $i = 0, 1, \dots, k-1$.
- A directed graph with no self-loops is **simple**.
- In an undirected graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $k \geq 3$ and $v_0 = v_k$; the cycle is **simple** if $v_1, v_2, \dots, v_k$ are distinct.

Defining Cycles

Definitions

- In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge.
- The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct.
- A self-loop is a simple cycle of length 1.
- Two paths $\langle v_0, v_1, \dots, v_{k-1}, v_0 \rangle$ and $\langle v'_0, v'_1, \dots, v'_{k-1}, v'_0 \rangle$ form the same cycle if there exists an integer j such that $v'_i = v_{i+j \bmod k}$ for $i = 0, 1, \dots, k-1$.
- A directed graph with no self-loops is **simple**.
- In an undirected graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $k \geq 3$ and $v_0 = v_k$; the cycle is **simple** if $v_1, v_2, \dots, v_k$ are distinct.
- A graph with no cycles is **acyclic**.

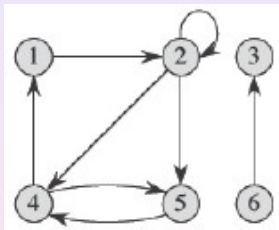
Cycle Examples



Explanation

- The path $\langle 1, 2, 4, 1 \rangle$ forms the same cycle as the paths $\langle 2, 4, 1, 2 \rangle$ and $\langle 4, 1, 2, 4 \rangle$.

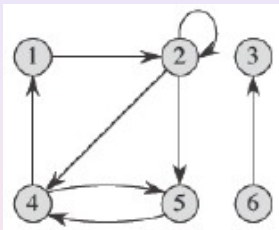
Cycle Examples



Explanation

- The path $\langle 1, 2, 4, 1 \rangle$ forms the same cycle as the paths $\langle 2, 4, 1, 2 \rangle$ and $\langle 4, 1, 2, 4 \rangle$.
- This cycle is simple but the cycle $\langle 1, 2, 4, 5, 4, 1 \rangle$ is not.

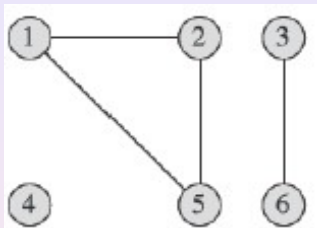
Cycle Examples



Explanation

- The path $\langle 1, 2, 4, 1 \rangle$ forms the same cycle as the paths $\langle 2, 4, 1, 2 \rangle$ and $\langle 4, 1, 2, 4 \rangle$.
- This cycle is simple but the cycle $\langle 1, 2, 4, 5, 4, 1 \rangle$ is not.
- The cycle $\langle 2, 2 \rangle$ formed by the edge $(2, 2)$ is a self-loop. Since the graph has a self-loop, it is not simple.

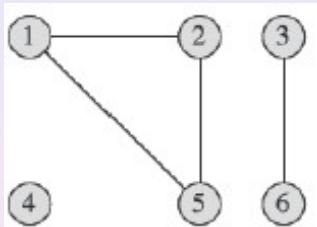
Connected Components



Definitions

- An undirected graph is **connected** if every vertex is reachable from all other vertices.

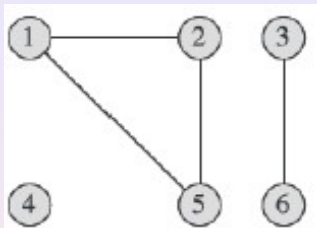
Connected Components



Definitions

- An undirected graph is **connected** if every vertex is reachable from all other vertices.
- The **connected components** of a graph are the equivalence classes of vertices under the "is reachable from" relation.

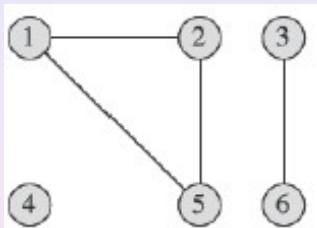
Connected Components



Definitions

- An undirected graph is **connected** if every vertex is reachable from all other vertices.
- The **connected components** of a graph are the equivalence classes of vertices under the "is reachable from" relation.
- The graph in the figure has three connected components: $\{1, 2, 5\}$, $\{3, 6\}$, and $\{4\}$.

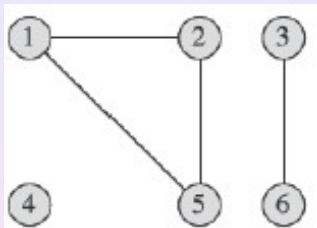
Connected Components



Definitions

- An undirected graph is **connected** if every vertex is reachable from all other vertices.
- The **connected components** of a graph are the equivalence classes of vertices under the "is reachable from" relation.
- The graph in the figure has three connected components: $\{1, 2, 5\}$, $\{3, 6\}$, and $\{4\}$.
- Every vertex in $\{1, 2, 5\}$ is reachable from every other vertex in $\{1, 2, 5\}$.

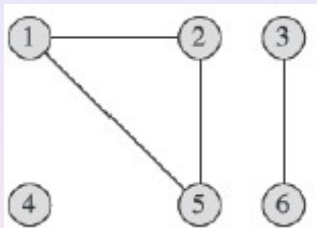
Connected Components



Definitions

- An undirected graph is **connected** if every vertex is reachable from all other vertices.
- The **connected components** of a graph are the equivalence classes of vertices under the "is reachable from" relation.
- The graph in the figure has three connected components: $\{1, 2, 5\}$, $\{3, 6\}$, and $\{4\}$.
- Every vertex in $\{1, 2, 5\}$ is reachable from every other vertex in $\{1, 2, 5\}$.
- An undirected graph is connected if it has exactly one connected component.

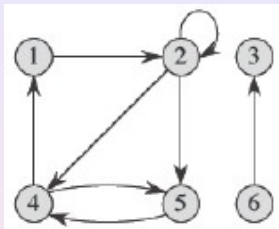
Connected Components



Definitions

- An undirected graph is **connected** if every vertex is reachable from all other vertices.
- The **connected components** of a graph are the equivalence classes of vertices under the "is reachable from" relation.
- The graph in the figure has three connected components: $\{1, 2, 5\}$, $\{3, 6\}$, and $\{4\}$.
- Every vertex in $\{1, 2, 5\}$ is reachable from every other vertex in $\{1, 2, 5\}$.
- An undirected graph is connected if it has exactly one connected component.
- The edges of a connected component are those that are incident on only the vertices of the component; in other words, edge (u, v) is an edge of a connected component only if both u and v are vertices of the component.

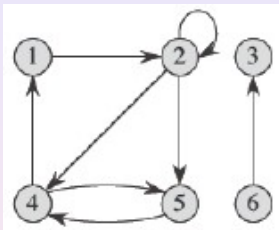
Strongly Connected Components



Definitions

- A directed graph is **strongly connected** if every two vertices are reachable from each other.

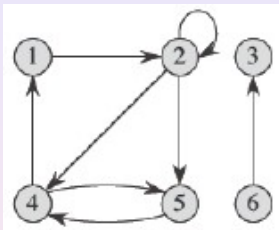
Strongly Connected Components



Definitions

- A directed graph is **strongly connected** if every two vertices are reachable from each other.
- The **strongly connected components** of a directed graph are the equivalence classes under the "are mutually reachable" relation.

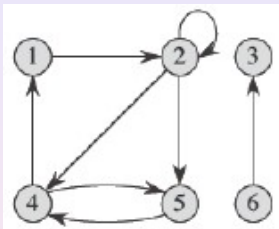
Strongly Connected Components



Definitions

- A directed graph is **strongly connected** if every two vertices are reachable from each other.
- The **strongly connected components** of a directed graph are the equivalence classes under the "are mutually reachable" relation.
- A directed graph is strongly connected if it has only one strongly connected component.

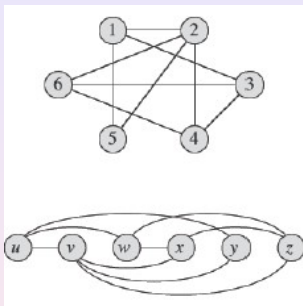
Strongly Connected Components



Definitions

- A directed graph is **strongly connected** if every two vertices are reachable from each other.
- The **strongly connected components** of a directed graph are the equivalence classes under the "are mutually reachable" relation.
- A directed graph is strongly connected if it has only one strongly connected component.
- The graph in the figure has three strongly connected components: $\{1, 2, 4, 5\}$, $\{3\}$, and $\{6\}$. The vertices $\{3, 6\}$ do not form a strongly connected component, since vertex 6 cannot be reached from vertex 3.

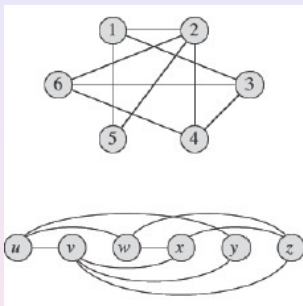
Graph Isomorphism



Definitions

- Two graphs $G = (V, E)$ and $G' = (V', E')$ are **isomorphic** if there exists a bijection $f : V \rightarrow V'$ such that $(u, v) \in E$ if and only if $(f(u), f(v)) \in E'$.

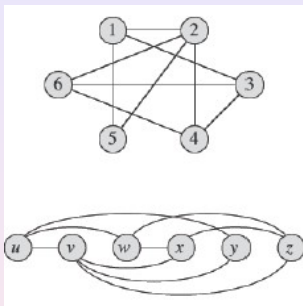
Graph Isomorphism



Definitions

- Two graphs $G = (V, E)$ and $G' = (V', E')$ are **isomorphic** if there exists a bijection $f : V \rightarrow V'$ such that $(u, v) \in E$ if and only if $(f(u), f(v)) \in E'$.
- In other words, we can relabel the vertices of G to be vertices of G' , maintaining the corresponding edges in G and G' .

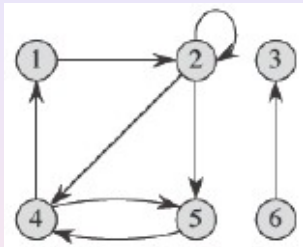
Graph Isomorphism



Definitions

- Two graphs $G = (V, E)$ and $G' = (V', E')$ are **isomorphic** if there exists a bijection $f : V \rightarrow V'$ such that $(u, v) \in E$ if and only if $(f(u), f(v)) \in E'$.
- In other words, we can relabel the vertices of G to be vertices of G' , maintaining the corresponding edges in G and G' .
- The figure shows a pair of isomorphic graphs G and G' . The mapping from V to V' given by $f(1) = u, f(2) = v, f(3) = w, f(4) = x, f(5) = y, f(6) = z$ provides the required bijective function.

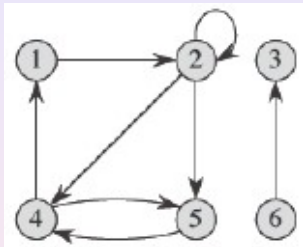
Subgraphs



Definitions

- We say that a graph $G' = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$.

Subgraphs

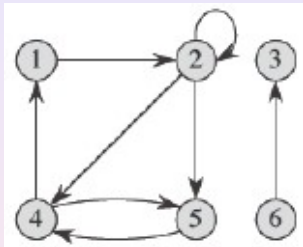


Definitions

- We say that a graph $G' = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$.
- Given a set $V' \subseteq V$, the subgraph of G **induced** by V' is the graph $G' = (V', E')$, where

$$E' = \{(u, v) \in E : u, v \in V'\}$$

Subgraphs



Definitions

- We say that a graph $G' = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$.
- Given a set $V' \subseteq V$, the subgraph of G **induced** by V' is the graph $G' = (V', E')$, where

$$E' = \{(u, v) \in E : u, v \in V'\}$$

- The subgraph induced by the vertex set $\{1, 2, 3, 6\}$ has the edge set $\{(1, 2), (2, 2), (6, 3)\}$.

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.
- That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version.

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.
- That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version.
- Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and $(u, v) \in E$.

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.
- That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version.
- Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and $(u, v) \in E$.
- That is, the undirected version contains the edges of G "with their directions removed" and with self-loops eliminated.

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.
- That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version.
- Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and $(u, v) \in E$.
- That is, the undirected version contains the edges of G "with their directions removed" and with self-loops eliminated.
- Since (u, v) and (v, u) are the same edge in an undirected graph, the undirected version of a directed graph contains it only once, even if the directed graph contains both edges (u, v) and (v, u) .

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.
- That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version.
- Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and $(u, v) \in E$.
- That is, the undirected version contains the edges of G "with their directions removed" and with self-loops eliminated.
- Since (u, v) and (v, u) are the same edge in an undirected graph, the undirected version of a directed graph contains it only once, even if the directed graph contains both edges (u, v) and (v, u) .
- In a directed graph $G = (V, E)$, a **neighbor** of a vertex u is any vertex that is adjacent to u in the undirected version of G . That is, v is a neighbor of u if $u \neq v$ and either $(u, v) \in E$ or $(v, u) \in E$.

Directed and Undirected Versions of Graphs

Definitions

- Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$.
- That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version.
- Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and $(u, v) \in E$.
- That is, the undirected version contains the edges of G "with their directions removed" and with self-loops eliminated.
- Since (u, v) and (v, u) are the same edge in an undirected graph, the undirected version of a directed graph contains it only once, even if the directed graph contains both edges (u, v) and (v, u) .
- In a directed graph $G = (V, E)$, a **neighbor** of a vertex u is any vertex that is adjacent to u in the undirected version of G . That is, v is a neighbor of u if $u \neq v$ and either $(u, v) \in E$ or $(v, u) \in E$.
- In an undirected graph, u and v are neighbors if they are adjacent.

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.
- A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$, or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 .

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.
- A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$, or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 .
- An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a **tree**. A directed acyclic graph is called a **dag**.

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.
- A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$, or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 .
- An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a **tree**. A directed acyclic graph is called a **dag**.
- There are two variants of graphs that you may occasionally encounter. A **multigraph** is like an undirected graph, but it can have both multiple edges between vertices and self-loops.

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.
- A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$, or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 .
- An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a **tree**. A directed acyclic graph is called a **dag**.
- There are two variants of graphs that you may occasionally encounter. A **multigraph** is like an undirected graph, but it can have both multiple edges between vertices and self-loops.
- A **hypergraph** is like an undirected graph, but each hyperedge, rather than connecting two vertices, connects an arbitrary subset of vertices.

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.
- A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$, or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 .
- An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a **tree**. A directed acyclic graph is called a **dag**.
- There are two variants of graphs that you may occasionally encounter. A **multigraph** is like an undirected graph, but it can have both multiple edges between vertices and self-loops.
- A **hypergraph** is like an undirected graph, but each hyperedge, rather than connecting two vertices, connects an arbitrary subset of vertices.
- We can represent a hypergraph by a bipartite graph if we let incidence in the hypergraph correspond to adjacency in the bipartite graph. That is, V_1 in the bipartite graph correspond to vertices of the hypergraph, and V_2 of the bipartite graph correspond to hyperedges.

Graphs with Special Names

Definitions

- A **complete graph** is an undirected graph in which every pair of vertices is adjacent.
- A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$, or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 .
- An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a **tree**. A directed acyclic graph is called a **dag**.
- There are two variants of graphs that you may occasionally encounter. A **multigraph** is like an undirected graph, but it can have both multiple edges between vertices and self-loops.
- A **hypergraph** is like an undirected graph, but each hyperedge, rather than connecting two vertices, connects an arbitrary subset of vertices.
- We can represent a hypergraph by a bipartite graph if we let incidence in the hypergraph correspond to adjacency in the bipartite graph. That is, V_1 in the bipartite graph correspond to vertices of the hypergraph, and V_2 of the bipartite graph correspond to hyperedges.
- The **contraction** of an undirected graph $G = (V, E)$ by an edge $e = (u, v)$ is a graph $G' = (V', E')$, where $V' = V - \{u, v\} \cup \{x\}$ and x is a new vertex. E' is formed from E by deleting (u, v) and, for each vertex w adjacent to u or v , deleting whichever of (u, w) and (v, w) is in E and adding the new edge (x, w) . In effect, u and v are "contracted" into a single vertex.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.
- Any two vertices in G are connected by a unique simple path.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.
- Any two vertices in G are connected by a unique simple path.
- G is connected, but if any edge is removed from E , the resulting graph is disconnected.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.
- Any two vertices in G are connected by a unique simple path.
- G is connected, but if any edge is removed from E , the resulting graph is disconnected.
- G is connected, and $|E| = |V| - 1$.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.
- Any two vertices in G are connected by a unique simple path.
- G is connected, but if any edge is removed from E , the resulting graph is disconnected.
- G is connected, and $|E| = |V| - 1$.
- G is acyclic, and $|E| = |V| - 1$.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.
- Any two vertices in G are connected by a unique simple path.
- G is connected, but if any edge is removed from E , the resulting graph is disconnected.
- G is connected, and $|E| = |V| - 1$.
- G is acyclic, and $|E| = |V| - 1$.
- G is acyclic, but if any edge is added to E , the resulting graph contains a cycle.

Trees

Alternative Definitions for Trees

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

- G is a tree.
- Any two vertices in G are connected by a unique simple path.
- G is connected, but if any edge is removed from E , the resulting graph is disconnected.
- G is connected, and $|E| = |V| - 1$.
- G is acyclic, and $|E| = |V| - 1$.
- G is acyclic, but if any edge is added to E , the resulting graph contains a cycle.



Figure: We have a tree on the left, and a forest in the middle. The graph on the right contains a cycle and is therefore neither a tree nor a forest.

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.
- If y is an **ancestor** of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y .

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.
- If y is an **ancestor** of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y .
- The **subtree rooted at x** is the tree induced by descendants of x , and rooted at x .

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.
- If y is an **ancestor** of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y .
- The **subtree rooted at x** is the tree induced by descendants of x , and rooted at x .
- If the last edge on the simple path from the root r of a tree T to a node x is (y, x) , then y is the **parent** of x , and x is a **child** of y .

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.
- If y is an **ancestor** of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y .
- The **subtree rooted at x** is the tree induced by descendants of x , and rooted at x .
- If the last edge on the simple path from the root r of a tree T to a node x is (y, x) , then y is the **parent** of x , and x is a **child** of y .
- The root is the only node in T with no parent.

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.
- If y is an **ancestor** of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y .
- The **subtree rooted at x** is the tree induced by descendants of x , and rooted at x .
- If the last edge on the simple path from the root r of a tree T to a node x is (y, x) , then y is the **parent** of x , and x is a **child** of y .
- The root is the only node in T with no parent.
- If two nodes have the same parent, they are **siblings**.

Rooted Trees

Definitions

- A **rooted tree** is a tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree.
- Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x .
- If y is an ancestor of x , then x is a **descendant** of y .
- Every node is both an ancestor and a descendant of itself.
- If y is an **ancestor** of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y .
- The **subtree rooted at x** is the tree induced by descendants of x , and rooted at x .
- If the last edge on the simple path from the root r of a tree T to a node x is (y, x) , then y is the **parent** of x , and x is a **child** of y .
- The root is the only node in T with no parent.
- If two nodes have the same parent, they are **siblings**.
- A node with no children is a **leaf** or **external node**. A nonleaf node is an **internal node**.

Alternative Ways To Represent Graphs

Adjacency Lists and Matrix

- There are two standard ways to represent a graph $G = (V, E)$: as a collection of *adjacency lists* or as an *adjacency matrix*. Either way applies to both directed and undirected graphs.

Alternative Ways To Represent Graphs

Adjacency Lists and Matrix

- There are two standard ways to represent a graph $G = (V, E)$: as a collection of *adjacency lists* or as an *adjacency matrix*. Either way applies to both directed and undirected graphs.
- Because the adjacency-list representation provides a compact way to represent *sparse* graphs, it is usually the method of choice.

Alternative Ways To Represent Graphs

Adjacency Lists and Matrix

- There are two standard ways to represent a graph $G = (V, E)$: as a collection of *adjacency lists* or as an *adjacency matrix*. Either way applies to both directed and undirected graphs.
- Because the adjacency-list representation provides a compact way to represent *sparse* graphs, it is usually the method of choice.
- Most graph algorithms assume that an input graph is represented in adjacency list form. We may prefer an adjacency-matrix representation when the graph is dense, or when we need to be able to tell quickly if there is an edge connecting two given vertices.

Alternative Ways To Represent Graphs

Adjacency Lists and Matrix

- There are two standard ways to represent a graph $G = (V, E)$: as a collection of *adjacency lists* or as an *adjacency matrix*. Either way applies to both directed and undirected graphs.
- Because the adjacency-list representation provides a compact way to represent *sparse* graphs, it is usually the method of choice.
- Most graph algorithms assume that an input graph is represented in adjacency list form. We may prefer an adjacency-matrix representation when the graph is dense, or when we need to be able to tell quickly if there is an edge connecting two given vertices.

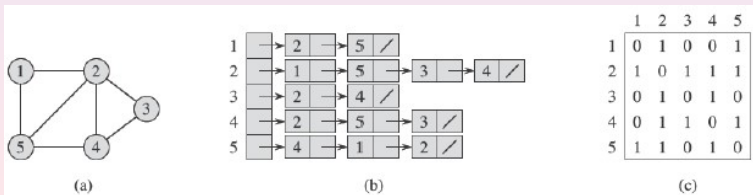


Figure: Representation of undirected graphs: (a) An undirected graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Directed Graphs and Adjacency Lists

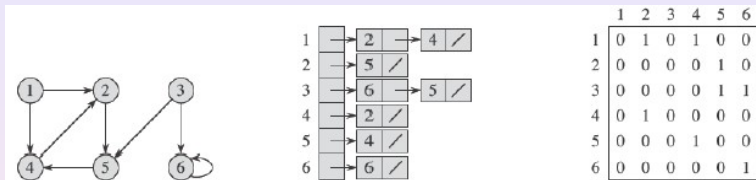


Figure: Representation of Directed graphs: (a) A directed graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Directed Graphs and Adjacency Lists

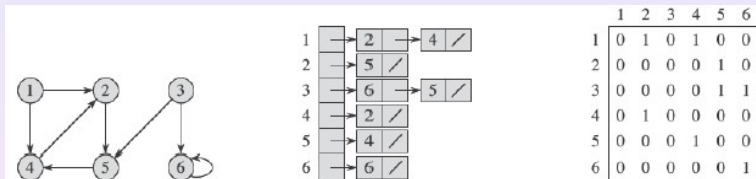


Figure: Representation of Directed graphs: (a) A directed graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Adjacency-List Representation and Memory Requirement

- **Adjacency-list representation** of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists.

Directed Graphs and Adjacency Lists

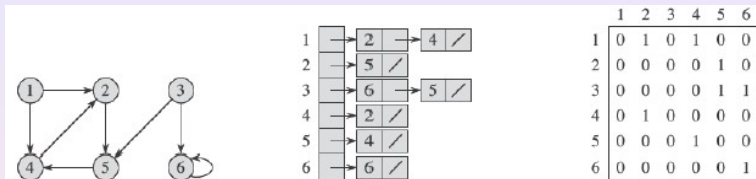


Figure: Representation of Directed graphs: (a) A directed graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Adjacency-List Representation and Memory Requirement

- **Adjacency-list representation** of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists.
- For each $u \in V$, $Adj[u]$ contains all the vertices (or pointers to vertices) **adjacent** to u in G .

Directed Graphs and Adjacency Lists

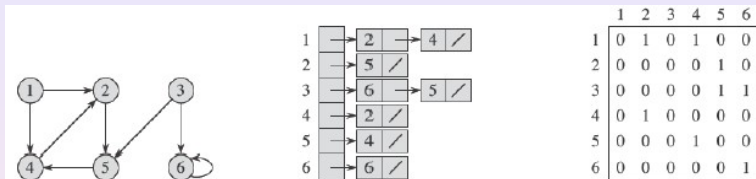


Figure: Representation of Directed graphs: (a) A directed graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Adjacency-List Representation and Memory Requirement

- **Adjacency-list representation** of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists.
- For each $u \in V$, $Adj[u]$ contains all the vertices (or pointers to vertices) **adjacent** to u in G .
- If G is a directed graph, the sum of the lengths of all the adjacency lists is $|E|$, since (u, v) is represented by having v appear in $Adj[u]$.

Directed Graphs and Adjacency Lists

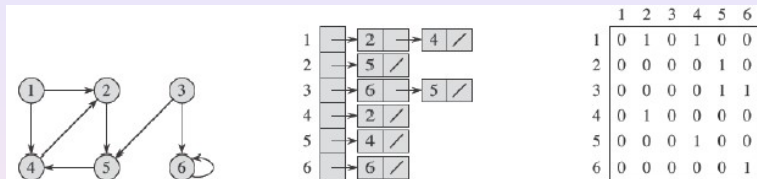


Figure: Representation of Directed graphs: (a) A directed graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Adjacency-List Representation and Memory Requirement

- **Adjacency-list representation** of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists.
- For each $u \in V$, $Adj[u]$ contains all the vertices (or pointers to vertices) **adjacent** to u in G .
- If G is a directed graph, the sum of the lengths of all the adjacency lists is $|E|$, since (u, v) is represented by having v appear in $Adj[u]$.
- If G is an undirected graph, the sum of the lengths of all the adjacency lists is $2|E|$, since if (u, v) is an undirected edge, then u appears in v 's adjacency list and vice versa.

Directed Graphs and Adjacency Lists

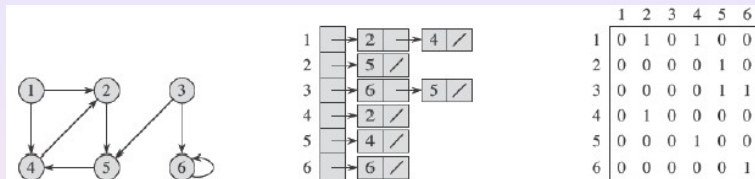


Figure: Representation of Directed graphs: (a) A directed graph G . (b) An adjacency list representation of G . (c) The adjacency matrix representation of G .

Adjacency-List Representation and Memory Requirement

- **Adjacency-list representation** of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists.
- For each $u \in V$, $Adj[u]$ contains all the vertices (or pointers to vertices) **adjacent** to u in G .
- If G is a directed graph, the sum of the lengths of all the adjacency lists is $|E|$, since (u, v) is represented by having v appear in $Adj[u]$.
- If G is an undirected graph, the sum of the lengths of all the adjacency lists is $2|E|$, since if (u, v) is an undirected edge, then u appears in v 's adjacency list and vice versa.
- For both cases, the adjacency-list representation has $\Theta(n + m)$ memory requirement.

Discussing Adjacency Lists

Weighted Graphs and Checking Existence of an Edge

- The graphs for which each edge has an associated weight, typically given by a weight function $w : E \rightarrow \mathbb{R}$, are called **weighted graphs**.

Discussing Adjacency Lists

Weighted Graphs and Checking Existence of an Edge

- The graphs for which each edge has an associated weight, typically given by a weight function $w : E \rightarrow \mathbb{R}$, are called **weighted graphs**.
- We can adapt adjacency lists to represent weighted graphs. We simply store the weight $w(u, v)$ of the edge (u, v) with vertex v in u 's adjacency list.

Discussing Adjacency Lists

Weighted Graphs and Checking Existence of an Edge

- The graphs for which each edge has an associated weight, typically given by a weight function $w : E \rightarrow \mathbb{R}$, are called **weighted graphs**.
- We can adapt adjacency lists to represent weighted graphs. We simply store the weight $w(u, v)$ of the edge (u, v) with vertex v in u 's adjacency list.
- A potential *disadvantage* of the adjacency-list representation is that it provides no quicker way to determine whether a given edge (u, v) is present in the graph than to search for v in the adjacency list $Adj[u]$.

Discussing Adjacency Lists

Weighted Graphs and Checking Existence of an Edge

- The graphs for which each edge has an associated weight, typically given by a weight function $w : E \rightarrow \mathbb{R}$, are called **weighted graphs**.
- We can adapt adjacency lists to represent weighted graphs. We simply store the weight $w(u, v)$ of the edge (u, v) with vertex v in u 's adjacency list.
- A potential *disadvantage* of the adjacency-list representation is that it provides no quicker way to determine whether a given edge (u, v) is present in the graph than to search for v in the adjacency list $Adj[u]$.
- Adjacency-matrix representation remedies this disadvantage, but at the cost of using asymptotically more memory.

Discussing Adjacency Lists

Weighted Graphs and Checking Existence of an Edge

- The graphs for which each edge has an associated weight, typically given by a weight function $w : E \rightarrow \mathbb{R}$, are called **weighted graphs**.
- We can adapt adjacency lists to represent weighted graphs. We simply store the weight $w(u, v)$ of the edge (u, v) with vertex v in u 's adjacency list.
- A potential *disadvantage* of the adjacency-list representation is that it provides no quicker way to determine whether a given edge (u, v) is present in the graph than to search for v in the adjacency list $Adj[u]$.
- Adjacency-matrix representation remedies this disadvantage, but at the cost of using asymptotically more memory.

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix of a graph requires $\Theta(n^2)$ memory, independent of the number of edges in the graph.

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix of a graph requires $\Theta(n^2)$ memory, independent of the number of edges in the graph.
- Since in an undirected graph, (u, v) and (v, u) represent the same edge, the adjacency matrix A of an undirected graph is its own transpose: $A = A^T$.

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix of a graph requires $\Theta(n^2)$ memory, independent of the number of edges in the graph.
- Since in an undirected graph, (u, v) and (v, u) represent the same edge, the adjacency matrix A of an undirected graph is its own transpose: $A = A^T$.
- To represent a weighted graph with the adjacency matrix representation, we can simply store the weight $w(u, v)$ of the edge (u, v) as the entry in row u and column v of the matrix.

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix of a graph requires $\Theta(n^2)$ memory, independent of the number of edges in the graph.
- Since in an undirected graph, (u, v) and (v, u) represent the same edge, the adjacency matrix A of an undirected graph is its own transpose: $A = A^T$.
- To represent a weighted graph with the adjacency matrix representation, we can simply store the weight $w(u, v)$ of the edge (u, v) as the entry in row u and column v of the matrix.
- If an edge does not exist, we can store a NIL value as its corresponding matrix entry, though for many problems it is convenient to use a value such as 0 or ∞ .

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix of a graph requires $\Theta(n^2)$ memory, independent of the number of edges in the graph.
- Since in an undirected graph, (u, v) and (v, u) represent the same edge, the adjacency matrix A of an undirected graph is its own transpose: $A = A^T$.
- To represent a weighted graph with the adjacency matrix representation, we can simply store the weight $w(u, v)$ of the edge (u, v) as the entry in row u and column v of the matrix.
- If an edge does not exist, we can store a NIL value as its corresponding matrix entry, though for many problems it is convenient to use a value such as 0 or ∞ .
- Although the adjacency-list representation is asymptotically at least as space efficient as the adjacency-matrix representation, adjacency matrices are simpler, and so we may prefer them when graphs are reasonably small.

Adjacency Matrix

Memory Requirement and Representing Weighted Graphs

- The **adjacency-matrix representation** of a graph $G = (V, E)$ consists of an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix of a graph requires $\Theta(n^2)$ memory, independent of the number of edges in the graph.
- Since in an undirected graph, (u, v) and (v, u) represent the same edge, the adjacency matrix A of an undirected graph is its own transpose: $A = A^T$.
- To represent a weighted graph with the adjacency matrix representation, we can simply store the weight $w(u, v)$ of the edge (u, v) as the entry in row u and column v of the matrix.
- If an edge does not exist, we can store a NIL value as its corresponding matrix entry, though for many problems it is convenient to use a value such as 0 or ∞ .
- Although the adjacency-list representation is asymptotically at least as space efficient as the adjacency-matrix representation, adjacency matrices are simpler, and so we may prefer them when graphs are reasonably small.
- Moreover, for unweighted graphs they require only one bit per entry.

Questions

Question 1

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where

$$E^T = \{(v, u) \in V \times V : (u, v) \in E\}.$$

Thus, G^T is G with all its edges reversed. What is the running time of computing G^T for both the adjacency-list and adjacency-matrix representations of G .

Questions

Question 1

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where

$$E^T = \{(v, u) \in V \times V : (u, v) \in E\}.$$

Thus, G^T is G with all its edges reversed. What is the running time of computing G^T for both the adjacency-list and adjacency-matrix representations of G .

Answer: $O(n + m)$ for adjacency-list, and $O(n^2)$ for adjacency-matrix.

Questions

Question 1

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where

$$E^T = \{(v, u) \in V \times V : (u, v) \in E\}.$$

Thus, G^T is G with all its edges reversed. What is the running time of computing G^T for both the adjacency-list and adjacency-matrix representations of G .

Answer: $O(n + m)$ for adjacency-list, and $O(n^2)$ for adjacency-matrix.

Follow-up Question: What if we have adjacency lists and incidence lists of G ?

Questions

Question 1

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where

$$E^T = \{(v, u) \in V \times V : (u, v) \in E\}.$$

Thus, G^T is G with all its edges reversed. What is the running time of computing G^T for both the adjacency-list and adjacency-matrix representations of G .

Answer: $O(n + m)$ for adjacency-list, and $O(n^2)$ for adjacency-matrix.

Follow-up Question: What if we have adjacency lists and incidence lists of G ? $O(1)$.

Questions

Question 1

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where

$$E^T = \{(v, u) \in V \times V : (u, v) \in E\}.$$

Thus, G^T is G with all its edges reversed. What is the running time of computing G^T for both the adjacency-list and adjacency-matrix representations of G .

Answer: $O(n + m)$ for adjacency-list, and $O(n^2)$ for adjacency-matrix.

Follow-up Question: What if we have adjacency lists and incidence lists of G ? $O(1)$.

Question 2

Most graph algorithms that take an adjacency-matrix representation as input require $\Omega(n^2)$ -time.

Exception: How can you determine whether a directed graph G contains a **universal sink** (a vertex with in-degree $n - 1$ and out-degree 0) in $O(n)$ -time, given an adjacency matrix for G ?

Questions

Question 1

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where

$$E^T = \{(v, u) \in V \times V : (u, v \in E)\}.$$

Thus, G^T is G with all its edges reversed. What is the running time of computing G^T for both the adjacency-list and adjacency-matrix representations of G .

Answer: $O(n + m)$ for adjacency-list, and $O(n^2)$ for adjacency-matrix.

Follow-up Question: What if we have adjacency lists and incidence lists of G ? $O(1)$.

Question 2

Most graph algorithms that take an adjacency-matrix representation as input require $\Omega(n^2)$ -time.

Exception: How can you determine whether a directed graph G contains a **universal sink** (a vertex with in-degree $n - 1$ and out-degree 0) in $O(n)$ -time, given an adjacency matrix for G ?

Answer: Start by examining position $(1, 1)$ in the adjacency matrix. When examining position (i, j) ,

- if a 1 is encountered, examine position $(i + 1, j)$, and
- if a 0 is encountered, examine position $(i, j + 1)$.

Once either i or j is equal to n , check if vertex i is a universal sink.

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

- ➊ It computes the distance (smallest number of edges) from s to each reachable vertex.

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

- 1 It computes the distance (smallest number of edges) from s to each reachable vertex.
- 2 It produces a *breadth-first tree* with root s that contains all reachable vertices.

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

- 1 It computes the distance (smallest number of edges) from s to each reachable vertex.
- 2 It produces a *breadth-first tree* with root s that contains all reachable vertices.
- 3 For any vertex v reachable from s , the simple path in the breadth-first tree from s to v corresponds to a *shortest path* from s to v in G .

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

- 1 It computes the distance (smallest number of edges) from s to each reachable vertex.
- 2 It produces a *breadth-first tree* with root s that contains all reachable vertices.
- 3 For any vertex v reachable from s , the simple path in the breadth-first tree from s to v corresponds to a *shortest path* from s to v in G .
- 4 The algorithm works on both directed and undirected graphs.

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

- 1 It computes the distance (smallest number of edges) from s to each reachable vertex.
- 2 It produces a *breadth-first tree* with root s that contains all reachable vertices.
- 3 For any vertex v reachable from s , the simple path in the breadth-first tree from s to v corresponds to a *shortest path* from s to v in G .
- 4 The algorithm works on both directed and undirected graphs.

Strategy Used in BFS

- Breadth-first search is so named because it expands the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier. That is, the algorithm discovers all vertices at distance k from s before discovering any vertices at distance $k + 1$.

Introducing the BFS Algorithm

Properties of the BFS Algorithm

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth-first search (BFS) algorithm systematically explores the edges of G to *discover* every vertex reachable from s .

- 1 It computes the distance (smallest number of edges) from s to each reachable vertex.
- 2 It produces a *breadth-first tree* with root s that contains all reachable vertices.
- 3 For any vertex v reachable from s , the simple path in the breadth-first tree from s to v corresponds to a *shortest path* from s to v in G .
- 4 The algorithm works on both directed and undirected graphs.

Strategy Used in BFS

- Breadth-first search is so named because it expands the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier. That is, the algorithm discovers all vertices at distance k from s before discovering any vertices at distance $k + 1$.
- Prim's minimum-spanning tree algorithm and Dijkstra's single-source shortest-paths algorithm use similar ideas.

Explaining Execution of BFS by a Picture

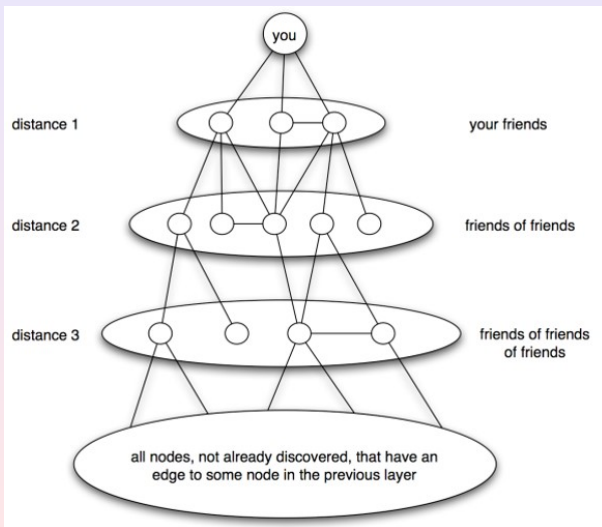


Figure: Breadth-First Search Algorithm

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.
- If $(u, v) \in E$ and vertex u is black, then vertex v is either gray or black; that is, all vertices adjacent to black vertices have been discovered.

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.
- If $(u, v) \in E$ and vertex u is black, then vertex v is either gray or black; that is, all vertices adjacent to black vertices have been discovered.
- Gray vertices may have some adjacent white vertices; they represent the frontier between discovered and undiscovered vertices.

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.
- If $(u, v) \in E$ and vertex u is black, then vertex v is either gray or black; that is, all vertices adjacent to black vertices have been discovered.
- Gray vertices may have some adjacent white vertices; they represent the frontier between discovered and undiscovered vertices.

BFS Tree

- BFS constructs a breadth-first tree, initially containing only its root vertex s .

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.
- If $(u, v) \in E$ and vertex u is black, then vertex v is either gray or black; that is, all vertices adjacent to black vertices have been discovered.
- Gray vertices may have some adjacent white vertices; they represent the frontier between discovered and undiscovered vertices.

BFS Tree

- BFS constructs a breadth-first tree, initially containing only its root vertex s .
- Whenever the search discovers a white vertex v in the course of scanning the adjacency list of an already discovered vertex u , the vertex v and the edge (u, v) are added to the tree.

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.
- If $(u, v) \in E$ and vertex u is black, then vertex v is either gray or black; that is, all vertices adjacent to black vertices have been discovered.
- Gray vertices may have some adjacent white vertices; they represent the frontier between discovered and undiscovered vertices.

BFS Tree

- BFS constructs a breadth-first tree, initially containing only its root vertex s .
- Whenever the search discovers a white vertex v in the course of scanning the adjacency list of an already discovered vertex u , the vertex v and the edge (u, v) are added to the tree.
- We say that u is the **predecessor** or **parent** of v in the breadth-first tree. Since a vertex is discovered at most once, it has at most one parent.

Details of the Algorithm

Explaining Vertex Coloring

- To keep track of progress, breadth-first search colors each vertex white, gray, or black: All vertices start out white and may later become gray and then black.
- A vertex is *discovered* the first time it is encountered during the search, at which time it becomes gray. Gray and black vertices, therefore, have been discovered.
- If $(u, v) \in E$ and vertex u is black, then vertex v is either gray or black; that is, all vertices adjacent to black vertices have been discovered.
- Gray vertices may have some adjacent white vertices; they represent the frontier between discovered and undiscovered vertices.

BFS Tree

- BFS constructs a breadth-first tree, initially containing only its root vertex s .
- Whenever the search discovers a white vertex v in the course of scanning the adjacency list of an already discovered vertex u , the vertex v and the edge (u, v) are added to the tree.
- We say that u is the **predecessor** or **parent** of v in the breadth-first tree. Since a vertex is discovered at most once, it has at most one parent.
- Ancestor/descendant relationships in the BFS tree are defined relative to s : if u is on the path in the tree between s and v , then u is an ancestor of v and v is a descendant of u .

Implementation Details

Data Structures Used By The Algorithm

- The BFS algorithm assumes that the input graph $G = (V, E)$ is represented using adjacency lists.

Implementation Details

Data Structures Used By The Algorithm

- The BFS algorithm assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- It attaches several additional attributes to each vertex in the graph.

Implementation Details

Data Structures Used By The Algorithm

- The BFS algorithm assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- It attaches several additional attributes to each vertex in the graph.
- We store the color of each vertex $u \in V$ in the attribute $u.color$ and the predecessor of u in the attribute $u.\pi$.

Implementation Details

Data Structures Used By The Algorithm

- The BFS algorithm assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- It attaches several additional attributes to each vertex in the graph.
- We store the color of each vertex $u \in V$ in the attribute $u.color$ and the predecessor of u in the attribute $u.\pi$.
- If u has no predecessor (for example, if $u = s$ or u has not been discovered), then $u.\pi = NIL$.

Implementation Details

Data Structures Used By The Algorithm

- The BFS algorithm assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- It attaches several additional attributes to each vertex in the graph.
- We store the color of each vertex $u \in V$ in the attribute $u.color$ and the predecessor of u in the attribute $u.\pi$.
- If u has no predecessor (for example, if $u = s$ or u has not been discovered), then $u.\pi = NIL$.
- The attribute $u.d$ holds the distance from the source s to vertex u computed by the algorithm.

Implementation Details

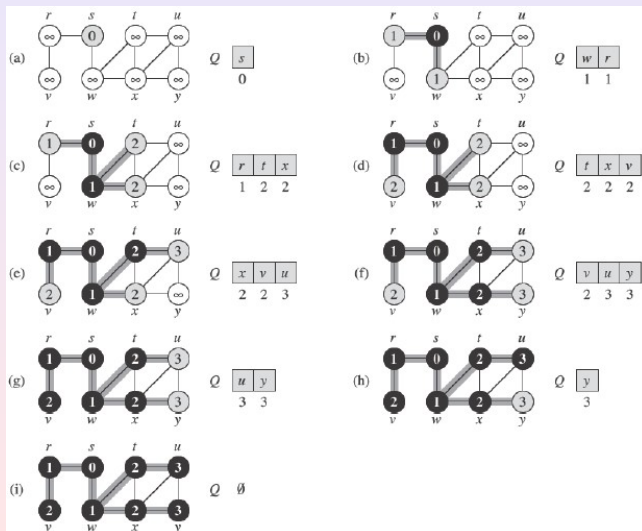
Data Structures Used By The Algorithm

- The BFS algorithm assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- It attaches several additional attributes to each vertex in the graph.
- We store the color of each vertex $u \in V$ in the attribute $u.color$ and the predecessor of u in the attribute $u.\pi$.
- If u has no predecessor (for example, if $u = s$ or u has not been discovered), then $u.\pi = NIL$.
- The attribute $u.d$ holds the distance from the source s to vertex u computed by the algorithm.
- The algorithm also uses a first-in, first-out queue Q to manage the set of gray vertices.

Pseudocode For The BFS Algorithm

```
BFS( $G, s$ )  
1  for each vertex  $u \in G.V - \{s\}$   
2       $u.color = \text{WHITE}$   
3       $u.d = \infty$   
4       $u.\pi = \text{NIL}$   
5   $s.color = \text{GRAY}$   
6   $s.d = 0$   
7   $s.\pi = \text{NIL}$   
8   $Q = \emptyset$   
9  ENQUEUE( $Q, s$ )  
10 while  $Q \neq \emptyset$   
11      $u = \text{DEQUEUE}(Q)$   
12     for each  $v \in G.Adj[u]$   
13         if  $v.color == \text{WHITE}$   
14              $v.color = \text{GRAY}$   
15              $v.d = u.d + 1$   
16              $v.\pi = u$   
17             ENQUEUE( $Q, v$ )  
18      $u.color = \text{BLACK}$ 
```


A Sample Run of the BFS Algorithm



Running-Time Analysis

Running-Time Analysis of BFS

- After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once.

Running-Time Analysis

Running-Time Analysis of BFS

- After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once.
- The operations of enqueueing and dequeuing take $O(1)$ time, and so the total time devoted to queue operations is $O(n)$.

Running-Time Analysis

Running-Time Analysis of BFS

- After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once.
- The operations of enqueueing and dequeuing take $O(1)$ time, and so the total time devoted to queue operations is $O(n)$.
- Because the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once.

Running-Time Analysis

Running-Time Analysis of BFS

- After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once.
- The operations of enqueueing and dequeueing take $O(1)$ time, and so the total time devoted to queue operations is $O(n)$.
- Because the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once.
- Since the sum of the lengths of all the adjacency lists is $\Theta(m)$, the total time spent in scanning adjacency lists is $O(m)$.

Running-Time Analysis

Running-Time Analysis of BFS

- After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once.
- The operations of enqueueing and dequeueing take $O(1)$ time, and so the total time devoted to queue operations is $O(n)$.
- Because the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once.
- Since the sum of the lengths of all the adjacency lists is $\Theta(m)$, the total time spent in scanning adjacency lists is $O(m)$.
- The overhead for initialization is $O(n)$, and thus the total running time of the BFS procedure is $O(n + m)$.

Running-Time Analysis

Running-Time Analysis of BFS

- After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once.
- The operations of enqueueing and dequeueing take $O(1)$ time, and so the total time devoted to queue operations is $O(n)$.
- Because the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once.
- Since the sum of the lengths of all the adjacency lists is $\Theta(m)$, the total time spent in scanning adjacency lists is $O(m)$.
- The overhead for initialization is $O(n)$, and thus the total running time of the BFS procedure is $O(n + m)$.
- Thus, breadth-first search runs in time linear in the size of the adjacency-list representation of the given graph.

Shortest Paths

Definitions

Define the **shortest-path distance** $\delta(s, v)$ from s to v as the minimum number of edges in any path from vertex s to vertex v ; if there is no path from s to v , then $\delta(s, v) = \infty$.

Shortest Paths

Definitions

Define the **shortest-path distance** $\delta(s, v)$ from s to v as the minimum number of edges in any path from vertex s to vertex v ; if there is no path from s to v , then $\delta(s, v) = \infty$.

We call a path of length $\delta(s, v)$ from s to v a **shortest path** from s to v .

Shortest Paths

Definitions

Define the **shortest-path distance** $\delta(s, v)$ from s to v as the minimum number of edges in any path from vertex s to vertex v ; if there is no path from s to v , then $\delta(s, v) = \infty$.

We call a path of length $\delta(s, v)$ from s to v a **shortest path** from s to v .

Lemma

Let $G = (V, E)$ be a directed or an undirected graph, and let $s \in V$ be an arbitrary vertex. Then, for any edge $(u, v) \in E$,

$$\delta(s, v) \leq \delta(s, u) + 1.$$

Shortest Paths

Definitions

Define the **shortest-path distance** $\delta(s, v)$ from s to v as the minimum number of edges in any path from vertex s to vertex v ; if there is no path from s to v , then $\delta(s, v) = \infty$.

We call a path of length $\delta(s, v)$ from s to v a **shortest path** from s to v .

Lemma

Let $G = (V, E)$ be a directed or an undirected graph, and let $s \in V$ be an arbitrary vertex. Then, for any edge $(u, v) \in E$,

$$\delta(s, v) \leq \delta(s, u) + 1.$$

Theorem

Let $G = (V, E)$ be a directed or an undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then upon termination, for each vertex $v \in V$, the value $v.d$ computed by BFS is equal to $\delta(s, v)$.

Predecessor Subgraph

Breadth-First Tree

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph** of G as $G_\pi = (V_\pi, E_\pi)$, where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

Predecessor Subgraph

Breadth-First Tree

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph** of G as $G_\pi = (V_\pi, E_\pi)$, where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

- The predecessor subgraph G_π is a **breadth-first tree**, that is,

Predecessor Subgraph

Breadth-First Tree

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph** of G as $G_\pi = (V_\pi, E_\pi)$, where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

- The predecessor subgraph G_π is a **breadth-first tree**, that is,
 - 1 V_π consists of the vertices reachable from s and,

Predecessor Subgraph

Breadth-First Tree

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph** of G as $G_\pi = (V_\pi, E_\pi)$, where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

- The predecessor subgraph G_π is a **breadth-first tree**, that is,
 - V_π consists of the vertices reachable from s and,
 - for all $v \in V_\pi$, the subgraph G_π contains a unique simple path from s to v that is also a shortest path from s to v in G .

Predecessor Subgraph

Breadth-First Tree

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph** of G as $G_\pi = (V_\pi, E_\pi)$, where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

- The predecessor subgraph G_π is a **breadth-first tree**, that is,
 - V_π consists of the vertices reachable from s and,
 - for all $v \in V_\pi$, the subgraph G_π contains a unique simple path from s to v that is also a shortest path from s to v in G .
- A breadth-first tree is in fact a tree, since it is connected and $|E_\pi| = |V_\pi| - 1$.

Predecessor Subgraph

Breadth-First Tree

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph** of G as $G_\pi = (V_\pi, E_\pi)$, where

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}, \text{ and}$$

$$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}.$$

- The predecessor subgraph G_π is a **breadth-first tree**, that is,
 - V_π consists of the vertices reachable from s and,
 - for all $v \in V_\pi$, the subgraph G_π contains a unique simple path from s to v that is also a shortest path from s to v in G .
- A breadth-first tree is in fact a tree, since it is connected and $|E_\pi| = |V_\pi| - 1$.
- The edges in E_π are called **tree edges**.

Printing A Shortest Path from s to a Vertex v

```
PRINT-PATH( $G, s, v$ )  
1  if  $v == s$   
2      print  $s$   
3  elseif  $v.\pi == \text{NIL}$   
4      print "no path from"  $s$  "to"  $v$  "exists"  
5  else PRINT-PATH( $G, s, v.\pi$ )  
6      print  $v$ 
```

Figure: The procedure prints out the vertices on a shortest path from s to v , assuming that BFS has already computed a breadth-first tree. It runs in time linear in the number of vertices in the path printed.

Definitions

Bipartite Graphs

An undirected graph is called **bipartite** if its node set V can be partitioned into sets X and Y in such a way that every edge has one endpoint in X and the other end in Y .

Definitions

Bipartite Graphs

An undirected graph is called **bipartite** if its node set V can be partitioned into sets X and Y in such a way that every edge has one endpoint in X and the other end in Y .

Graph Coloring

An undirected graph $G = (V, E)$ is called **k-colorable** if each vertex $v \in V$ can be assigned a color from a set of k colors such that for each edge $(u, v) \in E$, the colors of u and v are different.

Definitions

Bipartite Graphs

An undirected graph is called **bipartite** if its node set V can be partitioned into sets X and Y in such a way that every edge has one endpoint in X and the other end in Y .

Graph Coloring

An undirected graph $G = (V, E)$ is called **k-colorable** if each vertex $v \in V$ can be assigned a color from a set of k colors such that for each edge $(u, v) \in E$, the colors of u and v are different.

Odd-length Cycle Freeness

An undirected graph $G = (V, E)$ is called **odd-length cycle free** if it does not contain an odd-length cycle.

Proving Equivalence of Definitions

Theorem

Let $G = (V, E)$ be an undirected graph. Each one of the following 3 statements imply the other 2.

Proving Equivalence of Definitions

Theorem

Let $G = (V, E)$ be an undirected graph. Each one of the following 3 statements imply the other 2.

- ➊ G is bipartite.
- ➋ G is 2-colorable.
- ➌ G does not have an odd-length cycle.

Proving Equivalence of Definitions

Theorem

Let $G = (V, E)$ be an undirected graph. Each one of the following 3 statements imply the other 2.

- ❶ G is bipartite.
- ❷ G is 2-colorable.
- ❸ G does not have an odd-length cycle.

A Partial Proof

- $(1 \rightarrow 2)$: Color the vertices in X with white and the vertices in Y with black.

Proving Equivalence of Definitions

Theorem

Let $G = (V, E)$ be an undirected graph. Each one of the following 3 statements imply the other 2.

- ❶ G is bipartite.
- ❷ G is 2-colorable.
- ❸ G does not have an odd-length cycle.

A Partial Proof

- $(1 \rightarrow 2)$: Color the vertices in X with white and the vertices in Y with black.
- $(2 \rightarrow 3)$: Assume there is an odd-length cycle. But then there is an odd number of vertices on this cycle. Two consecutive vertices along the path has to have the same color, which contradicts G being 2-colorable.

Proving Equivalence of Definitions

Theorem

Let $G = (V, E)$ be an undirected graph. Each one of the following 3 statements imply the other 2.

- ❶ G is bipartite.
- ❷ G is 2-colorable.
- ❸ G does not have an odd-length cycle.

A Partial Proof

- $(1 \rightarrow 2)$: Color the vertices in X with white and the vertices in Y with black.
- $(2 \rightarrow 3)$: Assume there is an odd-length cycle. But then there is an odd number of vertices on this cycle. Two consecutive vertices along the path has to have the same color, which contradicts G being 2-colorable.

Proving Equivalence of Definitions

Theorem

Let $G = (V, E)$ be an undirected graph. Each one of the following 3 statements imply the other 2.

- ❶ G is bipartite.
- ❷ G is 2-colorable.
- ❸ G does not have an odd-length cycle.

A Partial Proof

- $(1 \rightarrow 2)$: Color the vertices in X with white and the vertices in Y with black.
- $(2 \rightarrow 3)$: Assume there is an odd-length cycle. But then there is an odd number of vertices on this cycle. Two consecutive vertices along the path has to have the same color, which contradicts G being 2-colorable.

Lemma

Let $G = (V, E)$ be an **undirected** graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Let $(u, v) \in E$ be a *nontree* edge. Then, $|u.d - v.d| \leq 1$.

Completing The Proof

Proof Continued

- $(3 \rightarrow 1)$: G does not have odd-length cycle and we need is to show that G is bipartite.

Completing The Proof

Proof Continued

- **(3 $\rightarrow$ 1)** : G does not have odd-length cycle and we need is to show that G is bipartite.
- Run the BFS algorithm starting from any vertex $s \in V$ and obtain the BFS tree.

Completing The Proof

Proof Continued

- **(3 $\rightarrow$ 1)** : G does not have odd-length cycle and we need to show that G is bipartite.
- Run the BFS algorithm starting from any vertex $s \in V$ and obtain the BFS tree.
- Let $(u, v) \in E$ be any nontree edge of G . By the above lemma, we know that $|u.d - v.d| \leq 1$, i.e., either u and v are in the same level, or one of u and v is one level above the other.

Completing The Proof

Proof Continued

- **(3 $\rightarrow$ 1)** : G does not have odd-length cycle and we need to show that G is bipartite.
- Run the BFS algorithm starting from any vertex $s \in V$ and obtain the BFS tree.
- Let $(u, v) \in E$ be any nontree edge of G . By the above lemma, we know that $|u.d - v.d| \leq 1$, i.e., either u and v are in the same level, or one of u and v is one level above the other.
- Assume that $u.d = v.d$. Let z be the lowest common ancestor of u and v in the BFS tree. Consider the following cycle: The path from u to z along the tree edges concatenated with the path from z to v along the tree edges concatenated with the edge (v, u) . This path has odd-length, which contradicts with G being odd-length cycle free.

Completing The Proof

Proof Continued

- **(3 $\rightarrow$ 1)** : G does not have odd-length cycle and we need is to show that G is bipartite.
- Run the BFS algorithm starting from any vertex $s \in V$ and obtain the BFS tree.
- Let $(u, v) \in E$ be any nontree edge of G . By the above lemma, we know that $|u.d - v.d| \leq 1$, i.e., either u and v are in the same level, or one of u and v is one level above the other.
- Assume that $u.d = v.d$. Let z be the lowest common ancestor of u and v in the BFS tree. Consider the following cycle: The path from u to z along the tree edges concatenated with the path from z to v along the tree edges concatenated with the edge (v, u) . This path has odd-length, which contradicts with G being odd-length cycle free.
- Thus, for any nontree edge (u, v) in E , we have $|u.d - v.d| = 1$.

Completing The Proof

Proof Continued

- **(3 $\rightarrow$ 1) :** G does not have odd-length cycle and we need is to show that G is bipartite.
- Run the BFS algorithm starting from any vertex $s \in V$ and obtain the BFS tree.
- Let $(u, v) \in E$ be any nontree edge of G . By the above lemma, we know that $|u.d - v.d| \leq 1$, i.e., either u and v are in the same level, or one of u and v is one level above the other.
- Assume that $u.d = v.d$. Let z be the lowest common ancestor of u and v in the BFS tree. Consider the following cycle: The path from u to z along the tree edges concatenated with the path from z to v along the tree edges concatenated with the edge (v, u) . This path has odd-length, which contradicts with G being odd-length cycle free.
- Thus, for any nontree edge (u, v) in E , we have $|u.d - v.d| = 1$.
- Let X consist of the vertices whose distance to s is even, and Y consist of the vertices whose distance to s is odd. Since for any edge $(i, j) \in E$, we have $|i.d - j.d| = 1$, every edge has one endpoint in X and the other end in Y .